

An Edge-Based Multi-Task System with Tri-Modal Perception for Autonomous Driving

Nhat-Tuong Do-Tran¹, Du Do², Ba Hung Ngo³,
Vu-Hoang Tran^{4*}, and Trung-Hieu Nguyen^{4*}

¹ National University of Singapore, Singapore
tuongdotn@u.nus.edu

² FPT University, Vietnam
dudtnse183340@fpt.edu.vn

³ Thanh Dong University, Vietnam
hungnb@thanhdong.edu.vn

⁴ Ho Chi Minh City University of Technology and Engineering, Vietnam
{hoangtv, hieuntr}@hcmute.edu.vn

<https://github.com/dottrannhattuong/Multitasking-For-Autonomous>

Abstract. Real-time perception is essential for autonomous driving, yet deploying accurate deep learning models on resource-constrained edge devices remains challenging. Existing systems often employ separate single-task networks, which require substantial computational and memory resources. Moreover, vision-only perception can become unreliable under adverse illumination and weather conditions. In this paper, we propose an edge-based multi-task system that concurrently processes three complementary modalities: (1) *External-visual perception*, where a lightweight MobileNetV2 backbone with a BiFPN neck jointly performs object detection, lane detection, and driving-context classification; (2) *Spatial-environmental perception*, where a 2D LiDAR provides obstacle-distance measurements to complement visual object detection; and (3) *Internal-vehicle perception*, where the controller area network (CAN bus) provides real-time vehicle states such as speed and battery level.

Experimentally, the proposed multi-task visual model is first validated in a Unity simulation and on a scale-model vehicle. Subsequently, the complete tri-modal perception system is deployed on a real-world electric car (Renault Twizy) using an NVIDIA Jetson Nano. The multi-task visual network achieves 88.1% mAP@0.5 for object detection and 95.0% IoU for lane detection and operates at 14.63 FPS using FP16 TensorRT inference. Meanwhile, the LiDAR and CAN bus pipelines operate with processing cycles of approximately 55 ms and 2.3 ms, respectively. Our results demonstrate that tri-modal perception can be efficiently deployed on resource-constrained edge devices for real-time autonomous driving.

Keywords: Autonomous driving · Multi-task learning · Tri-modal perception · Edge computing · 2D LiDAR · CAN bus.

* denotes equal advising.

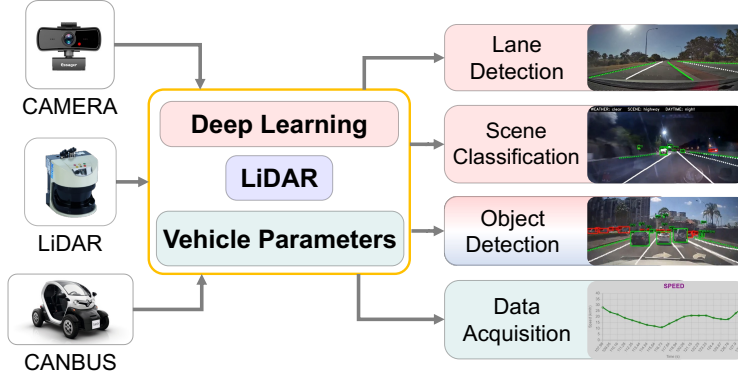


Fig. 1: **Overview of the proposed edge-based multi-task system with tri-modal perception.** **Left:** The camera captures external-visual information, the SICK LMS511-PRO 2D LiDAR [21] provides obstacle-distance measurements, and the CAN bus provides access to internal-vehicle information. **Center:** The multi-task visual network processes camera frames, the LiDAR pipeline measures obstacle distances, and the CAN bus pipeline acquires vehicle states. **Right:** The system produces lane-segmentation and driving-scene-classification outputs, an object-detection output complemented by LiDAR-based obstacle-distance measurements, and vehicle-state data.

1 Introduction

Autonomous driving has become an important application of artificial intelligence, with the potential to improve road safety, transportation efficiency, and mobility while reducing the need for human intervention. In recent years, many studies have focused on key perception tasks for autonomous driving, including lane detection [9, 15, 23, 33], object detection [2, 29, 34], and driving-scene classification [13, 26]. However, safe operation in dynamic environments requires an autonomous vehicle to perform several perception tasks accurately and in real time: detecting surrounding objects, understanding road structures, recognizing driving conditions, and monitoring its internal state. Delayed or incomplete perception can affect planning and control decisions, increasing the risk of accidents [11]. Therefore, an efficient and comprehensive perception system is essential for practical autonomous driving.

Despite recent advances, supporting multiple perception tasks on low-cost, resource-constrained edge devices presents two challenges. First, executing U-Net [17] for *lane detection*, YOLOv7 [29] for *object detection*, and ResNet-50 [5, 13] for *driving-scene classification* requires three independent feature extractors, causing redundant computation and high memory use. This workload is manageable on a workstation-class GPU such as the NVIDIA GeForce RTX 3090¹, which provides 10,496 CUDA cores. However, its 350W power consumption and cooling

¹ <https://www.nvidia.com/en-au/geforce/graphics-cards/30-series/rtx-3090-3090ti>

requirements hinder onboard deployment under a vehicle’s limited power budget. In contrast, the NVIDIA Jetson Nano² operates in a 10W power mode but provides only 128 CUDA cores. Lightweight backbones [6, 10, 24] and multi-task architectures [1, 14, 20, 27] offer practical ways to reduce computational cost. However, jointly trained tasks may produce conflicting gradients, causing optimization for one task to degrade the performance of another. Consequently, a shared multi-task model may achieve lower per-task accuracy than independently trained models [22, 31]. This raises **RQ1: How can a multi-task perception system achieve real-time throughput on a resource-constrained edge device while preserving per-task accuracy?**

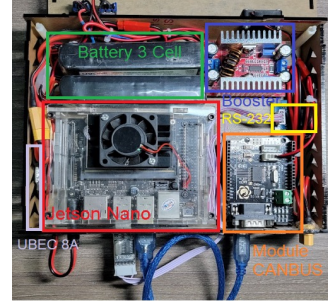
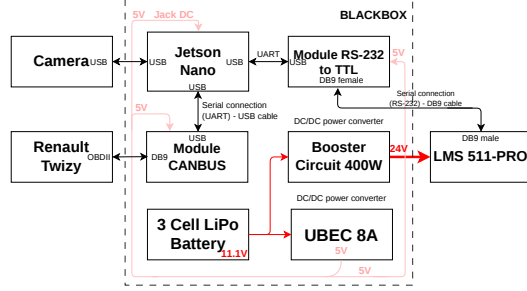
Second, camera-based deep models are influenced by their training distributions and may become unreliable when operating conditions differ substantially from those encountered during training, particularly under adverse illumination or weather [18, 32]. Moreover, monocular cameras do not directly measure metric obstacle distances and cannot access internal vehicle states. Consequently, camera-only models may provide incomplete or unreliable information for autonomous-driving perception. These limitations motivate **RQ2: How can LiDAR-based obstacle measurements and vehicle-state data acquired through the CAN bus complement camera-based perception?**

To address **RQ1** and **RQ2**, we propose an edge-based tri-modal perception system that captures three essential aspects of autonomous-driving perception. (1) The *external-visual perception* module understands the surrounding environment through object detection, lane detection, and driving-scene classification. A shared MobileNetV2 backbone [19] and BiFPN neck [25] replace three independent feature extractors, enabling task-specific heads to reuse shared features and thereby reducing computation while preserving competitive performance, addressing **RQ1**. As monocular visual perception primarily provides semantic scene information, (2) the *spatial-environmental perception* module complements it with LiDAR-based metric measurements of nearby obstacles. (3) The *internal-vehicle perception* module further acquires vehicle states through the CAN bus, including speed, state of charge, and motor speed (RPM). Together, these three pipelines provide semantic visual understanding, metric spatial sensing, and internal-state monitoring, enabling comprehensive perception on a resource-constrained edge device and addressing **RQ2**.

Our contributions are:

- We introduce a tri-modal edge perception system that concurrently processes camera-based semantics, LiDAR-derived metric measurements, and CAN-derived vehicle states on a Renault Twizy.
- We develop a shared multi-task visual model with FP16 TensorRT deployment on an NVIDIA Jetson Nano, enabling object detection, lane detection, and driving-scene classification within a single forward pass.
- We establish a progressive evaluation across Unity, a scale-model platform, and the Renault Twizy. The complete visual model achieves 88.1% mAP@0.5,

² <https://developer.nvidia.com/embedded/jetson-nano>



(a) Hardware configuration of the onboard tri-modal perception system. (b) Physical layout of the on-board hardware unit.

Fig. 2: **Onboard hardware integration.** (a) The camera connects to the Jetson Nano via USB, the LMS511-PRO LiDAR via RS-232, and the Renault Twizy CAN bus through an MCP2515-based interface. A three-cell 11.1 V LiPo battery supplies the system; an 8-A UBEC generates 5 V for the Jetson Nano and interfaces, whereas a DC-DC booster generates 24 V for the LiDAR. (b) These components are packaged into a compact black-box unit for onboard perception and telemetry without transmitting vehicle-control commands.

95.0% lane IoU, and 14.63 FPS, while the LiDAR and CAN pipelines operate with acquisition cycles of approximately 55 ms and 2.3 ms, respectively.

2 Methodology

System overview. The proposed system performs three complementary forms of perception on an NVIDIA Jetson Nano: external-visual perception from an RGB frame $I \in \mathbb{R}^{3 \times H \times W}$ captured by a camera; spatial-environmental perception from LiDAR data $\mathcal{S} = (s, o, \theta_0, \Delta\theta, \{d_i\}_{i=1}^N)$ acquired by a SICK LMS511-PRO 2D LiDAR³ [21]; and internal-vehicle perception from CAN data $\mathcal{C}$ received from the real-world Renault Twizy vehicle [16]. The three processing paths are written as:

$$(\hat{\mathcal{D}}, \hat{\mathcal{L}}, \hat{\mathbf{y}}) = f_{\phi}(I), \quad \mathcal{P} = g_L(\mathcal{S}), \quad \mathbf{z} = g_C(\mathcal{C}), \quad (1)$$

where the learned external-visual network f_{ϕ} performs object detection to produce $\hat{\mathcal{D}}$, lane-point estimation to produce $\hat{\mathcal{L}}$, and driving-scene classification to produce $\hat{\mathbf{y}}$. For spatial-environmental perception, g_L converts the LiDAR data $\mathcal{S}$ into the Cartesian point set $\mathcal{P}$ through telegram decoding followed by coordinate transformation. For internal-vehicle perception, g_C decodes the CAN data $\mathcal{C}$ into the vehicle-state vector $\mathbf{z}$. Following this functional organization, Fig. 2 presents the onboard hardware architecture alongside its physical black-box implementation.

³ SICK LMS511-10100 PRO

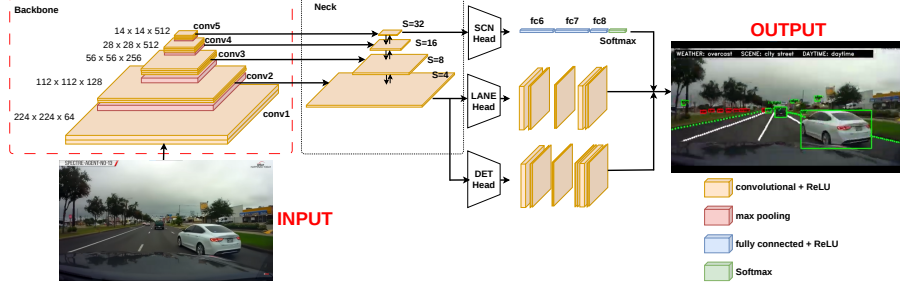


Fig. 3: **External-visual multi-task architecture.** A shared MobileNetV2 backbone extracts multi-scale features for BiFPN fusion. The stride-4 feature supports object detection and lane-point estimation, whereas the stride-32 feature supports weather, road-scene, and time-of-day classification.

2.1 External-Visual Perception

The visual branch processes each RGB frame through a shared MobileNetV2–BiFPN hierarchy followed by three prediction heads. As illustrated in Fig. 3, sharing the multi-scale representation reduces the computation and memory required by separate task-specific networks.

Shared Feature Extraction and Fusion. Given an RGB frame $I \in \mathbb{R}^{3 \times H \times W}$, the MobileNetV2 backbone B_{ϕ_b} [19] extracts feature maps at output strides $\mathcal{R} = \{4, 8, 16, 32\}$. The BiFPN neck N_{ϕ_n} [25] subsequently fuses these maps across scales:

$$\begin{aligned} \mathcal{F}_S &= B_{\phi_b}^{(S)}(I) \in \mathbb{R}^{C_S \times H/S \times W/S}, \quad S \in \mathcal{R}, \\ \{\mathcal{B}_S\}_{S \in \mathcal{R}} &= N_{\phi_n}(\{\mathcal{F}_S\}_{S \in \mathcal{R}}), \end{aligned} \quad (2)$$

where C_S denotes the number of channels at stride S . The fused hierarchy provides $\mathcal{B}_4$ to the spatial prediction heads, whereas $\mathcal{B}_{32}$ supports frame-level classification. Reusing this hierarchy eliminates repeated feature extraction across tasks. The task mappings are defined as:

$$\begin{aligned} \hat{\mathcal{D}} &= h_{\text{det}}(\mathcal{B}_4), \quad (\hat{\mathbf{H}}_L, \hat{\mathbf{O}}_L) = h_{\text{lane}}(\mathcal{B}_4), \\ \hat{\mathcal{L}} &= \Pi_L(\hat{\mathbf{H}}_L, \hat{\mathbf{O}}_L), \quad \hat{\mathbf{y}} = h_{\text{scene}}(\mathcal{B}_{32}). \end{aligned} \quad (3)$$

Multi-Task Prediction Heads. The outputs in eq. (3) provide three complementary levels of visual understanding:

- **Object Detection.** At the instance level, h_{det} maps $\mathcal{B}_4$ to the object set $\hat{\mathcal{D}}$. Each prediction contains a bounding box $\hat{\mathbf{b}}_n$, a predicted class $\hat{c}_n$, and a confidence score $\hat{s}_n$. High-resolution features support precise object localization.
- **Lane Detection.** At the structural level, h_{lane} reuses $\mathcal{B}_4$ to estimate a keypoint heatmap $\hat{\mathbf{H}}_L$ together with an association-offset map $\hat{\mathbf{O}}_L$. The decoder Π_L groups the recovered keypoints into $\hat{\mathcal{L}}$, where each $\hat{\mathcal{L}}_i$ represents a lane through ordered image coordinates $(\hat{x}_{i,q}, \hat{y}_{i,q})$.

- **Scene Classification.** At the contextual level, h_{scene} processes $\mathcal{B}_{32}$ to summarize the global conditions of the input frame. Its output $\hat{\mathbf{y}}$ contains class-probability vectors $\hat{\mathbf{p}}^a$ for $a \in \mathcal{A} = \{\text{weather, road scene, time of day}\}$.

Multi-Task Objective. Let $\mathcal{D}$, $\mathcal{L}$, and $\mathbf{y}$ denote the ground-truth annotations for the three tasks. The parameter set $\phi = \{\phi_b, \phi_n, \phi_{\text{det}}, \phi_{\text{lane}}, \phi_{\text{scene}}\}$ is optimized jointly using the following objective:

$$\begin{aligned}\mathcal{L}_{\text{det}} &= \mathcal{L}_{\text{bbox}} + \mathcal{L}_{\text{QFL}} + \mathcal{L}_{\text{DFL}}, \\ \mathcal{L}_{\text{lane}} &= \mathcal{L}_{\text{heat}}^{\text{lane}} + \eta \mathcal{L}_{\text{offset}}^{\text{lane}}, \\ \mathcal{L}_{\text{scene}} &= - \sum_{a \in \mathcal{A}} \sum_{c=1}^{K_a} y_c^a \log \hat{p}_c^a, \\ \mathcal{L}_{\text{vis}} &= \mathcal{L}_{\text{det}} + \mathcal{L}_{\text{lane}} + \mathcal{L}_{\text{scene}},\end{aligned}\tag{4}$$

where $\mathcal{L}_{\text{bbox}}$ is the IoU-based bounding-box regression loss, $\mathcal{L}_{\text{QFL}}$ is Quality Focal Loss, and $\mathcal{L}_{\text{DFL}}$ is Distribution Focal Loss. The terms $\mathcal{L}_{\text{heat}}^{\text{lane}}$ and $\mathcal{L}_{\text{offset}}^{\text{lane}}$ supervise lane-point localization and association, respectively, while K_a denotes the number of classes for attribute a .

2.2 Spatial-Environmental Perception

The spatial-environmental branch uses a SICK LMS511-PRO 2D LiDAR. The sensor supports a 190° field of view and is configured at 50 Hz with an angular increment of 0.5° . The implemented pipeline retains the forward sector from 55° to 125° , yielding $N = 141$ range samples per scan.

The Jetson Nano requests scan data according to the SICK telegram specification⁴. For each returned telegram, the parser locates the **DIST1** block and extracts the scale factor s , scale offset o , start angle θ_0 , angular increment $\Delta\theta$, number of measurements N , and hexadecimal range samples $\{d_i\}_{i=1}^N$. The corresponding range and bearing are:

$$r_i = s d_i + o, \quad \theta_i = \theta_0 + (i-1)\Delta\theta, \quad i = 1, \dots, N.\tag{5}$$

For the configuration used in this work, $s = 1$ and $o = 0$, so the decoded ranges are expressed directly in millimetres. After converting the telegram angles to radians, each polar return is mapped to the LiDAR Cartesian frame:

$$\mathbf{p}_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix} = \begin{bmatrix} r_i \cos \theta_i \\ r_i \sin \theta_i \end{bmatrix}, \quad \mathcal{P} = \{\mathbf{p}_i\}_{i=1}^N.\tag{6}$$

The LiDAR path complements external-visual perception with metric obstacle information unavailable from a monocular image. While $\hat{\mathcal{D}}$ provides class labels and image-plane locations, each $\mathbf{p}_i \in \mathcal{P}$ gives a range $r_i = \|\mathbf{p}_i\|_2$ at a known bearing. LiDAR can therefore retain evidence of an obstacle missed by the visual detector, as illustrated qualitatively in section 3.4. Such evidence indicates

⁴ SICK Telegram Listing LiDAR sensors (Part no. 8014631)

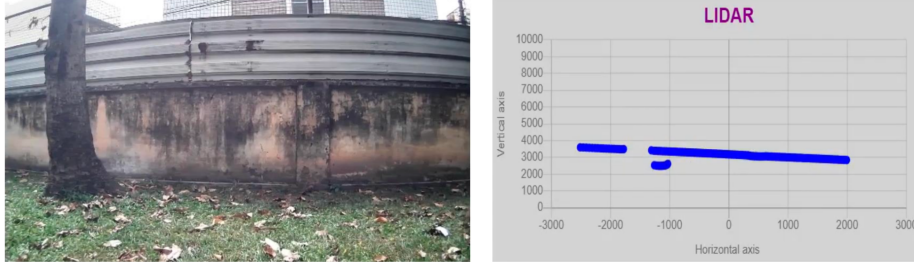


Fig. 4: **Complementary visual and spatial outputs.** **Left:** Camera view used for external-visual perception. **Right:** Cartesian LiDAR point set providing metric obstacle ranges and spatial distribution. No image–LiDAR projection or point-to-object association is applied.

obstacle presence and distance, but not its semantic identity. The outputs remain independent: no image projection, point-to-box association, or feature-level fusion is performed.

The LiDAR communicates with the Jetson Nano through an RS-232 interface configured at 230,400 bit/s using 8-N-1 framing. The selected telegram comprises an 81-byte header, 141 five-byte measurements, and a 12-byte trailer. Since each transmitted byte occupies ten bits, the telegram size and ideal UART transfer time are:

$$B_{\text{tel}} = 81 + 141 \times 5 + 12 = 798 \text{ bytes},$$

$$T_{\text{UART}} = \frac{10B_{\text{tel}}}{230,400} \approx 34.6 \text{ ms.} \quad (7)$$

Together with the 20 ms scan interval, this gives an estimated cycle of 54.6 ms, consistent with the measured value of approximately 55 ms. Here, T_{UART} covers serialization only, excluding parsing and visualization.

Fig. 4 presents the camera view alongside the Cartesian point set generated by eq. (6), illustrating the complementary visual and metric descriptions produced by the two perception paths.

2.3 Internal-Vehicle Perception

The internal-vehicle branch acquires Renault Twizy telemetry through the OBD-II connector [3]. The CAN network operates at 1 Mbit/s and is accessed using an MCP2515 controller⁵ with a TJA1050 transceiver⁶. An Arduino Uno filters relevant frames, represented by $\mathbf{c} = (q, \mathbf{b}, t^C)$ with $\mathbf{b} \in \{0, \dots, 255\}^\ell$ and $\ell \leq 8$, where q , $\mathbf{b}$, and t^C denote the identifier, payload, and reception timestamp, respectively. An identifier-specific decoder ψ_q applies the required scale factors, offsets, and status mappings.

⁵ <https://www.microchip.com/en-us/product/MCP2515>

⁶ <https://www.nxp.com/products/TJA1050>

Table 1: Field order of the 15-byte UART telemetry record.

No. State	No. State	No. State
1 Motor speed (RPM)	6 Battery voltage	11 Motor status
2 Vehicle speed	7 Vehicle status	12 Throttle position
3 Odometer	8 State of charge	13 Required current
4 Motor temperature	9 Remaining distance	14 State of health
5 Gear	10 Battery temperature	15 Brake status

Eight identifiers are retained: 0x19F for motor and vehicle speed, 0x196 for motor temperature, 0x59B for gear, throttle, motor status, and brake status, 0x155 for required current and state of charge, 0x424 for battery temperature and state of health, 0x425 for vehicle status, 0x599 for the odometer, and 0x55F for battery voltage. The decoded vehicle, motor, and battery states are packed into the fixed 15-byte UART record summarized in Tab. 1.

The Arduino transmits this record to the Jetson Nano through a 115,200-baud 8-N-1 UART link. Since each byte requires ten transmitted bits, the ideal serialization time is

$$T_{\text{UART}} = \frac{10B_{\text{rec}}}{R_{\text{UART}}} = \frac{150}{115,200} = 1.302 \text{ ms}, \quad (8)$$

where $B_{\text{rec}} = 15$ bytes and $R_{\text{UART}} = 115,200$ bit/s. This value represents only Arduino–Jetson serialization, not CAN bus latency. The CAN path is receive-only and transmits no vehicle-control frames.

2.4 User Interface

We implement the web interface shown in Fig. 6 to consolidate the three perception streams for system monitoring. We use Flask⁷ to provide the API endpoints and HTML, CSS, and JavaScript to render a three-panel dashboard. The left panel presents CAN-derived vehicle, motor, and battery states together with service controls; the upper-right panel displays object detections, lane overlays, and weather, road-scene, and time-of-day predictions; and the lower-right panel visualizes LiDAR measurements and vehicle-speed history.

Given the asynchronous update rates, we denote the output sequence of modality $r \in \{V, L, C\}$ as $\{(\mathbf{o}_{r,n}, t_{r,n})\}_n$. At interface time t , we select the latest sample from each modality and form the displayed output as:

$$\pi_r(t) = \max\{n : t_{r,n} \leq t\}, \quad \mathbf{u}(t) = [\mathbf{o}_{r,\pi_r(t)}]_{r \in \{V,L,C\}}. \quad (9)$$

Consequently, $\mathbf{u}(t)$ combines the latest available output from each modality rather than a temporally aligned sensor tuple. The LiDAR and CAN panels are updated at 1 Hz. The interface controls only the monitoring and inference services and issues no vehicle-actuation commands.

⁷ <https://flask.palletsprojects.com/>

3 Experiments

3.1 Experimental Setting

Dataset. The dataset contains 27,000 original images, comprising 15,000 images generated using the CARLA simulator [4], 10,000 images generated using a Unity-based simulation environment [28], and 2,000 images captured using the scale-model vehicle. These images were first divided into 21,000 training and 6,000 test samples. Data augmentation was then applied *exclusively to the training set*, preventing augmented variants of the training images from entering the test set. This process generated 3,000 additional samples, yielding a final dataset of 24,000 training and 6,000 test images. The photometric transformations adjust colour, saturation, brightness, and contrast, whereas the geometric transformations include rotation, scaling, cropping, stretching, and translation.

Implementation and Metrics. Visual-model training was conducted on an NVIDIA GeForce RTX 3090. The proposed multi-task visual network was trained for 75 epochs using the Adam optimizer with a batch size of 64. Accuracy was evaluated on the 6,000-image test set, while onboard deployability and FPS were evaluated separately on an NVIDIA Jetson Nano. The proposed model was accelerated using FP16 TensorRT. Lane detection was evaluated using intersection over union (IoU), whereas object detection was evaluated using mAP@0.5. Scene classification, progressive validation, and tri-modal operation were assessed qualitatively, while the LiDAR and CAN pipelines were characterized by their processing-cycle times.

3.2 Progressive Experimental Validation

The proposed system is progressively validated from simulation to physical deployment. First, the multi-task visual model is evaluated using images generated in Unity under controlled environmental conditions. It is subsequently tested on the scale-model platform, which introduces physical camera effects, illumination variations, and vehicle motion. Finally, the external-visual, LiDAR, and CAN pipelines are deployed together on the Renault Twizy for passive data acquisition and onboard processing, without transmitting vehicle-control commands.

Representative outputs from the three stages are shown in Fig. 5. Object detections, lane points, and scene labels remain observable as the evaluation progresses from simulation to the scale-model platform and real vehicle. In the scale-model example, the weather head returns an undefined label while the lane and object heads continue to produce outputs, illustrating that individual tasks may respond differently to the same input condition. These examples verify functional operation across the three platforms but do not constitute a quantitative comparison of cross-environment accuracy.

3.3 Lane-Detection Results

The results in Tab. 2a show clear accuracy-throughput differences among the evaluated models on the Jetson Nano. U-Net [17] achieves the highest standalone

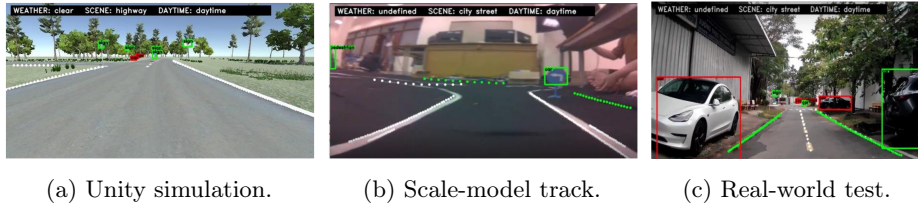


Fig. 5: **Progressive experimental validation.** The results demonstrate multi-task perception across increasingly realistic operating conditions. Bounding boxes indicate detected objects, colored points represent estimated lanes, and the top banner reports predicted weather, road scene, and time of day.

Table 2: **Accuracy and onboard throughput comparison for lane perception and object detection.** FPS values were measured on the Jetson Nano. Ours jointly performs lane perception, object detection, and scene classification. *Failed* indicates an inference runtime failure on the Jetson Nano.

(a) Lane-detection comparison.			(b) Object-detection comparison.		
Model	IoU (%) $\uparrow$	FPS $\uparrow$	Model	mAP ₅₀ (%) $\uparrow$	FPS $\uparrow$
U-Net [17]	94.8	<i>Failed</i>	YOLOv5m [7]	86.1	3.80
BiSeNet [30]	92.1	6.40	YOLOv6n [8]	85.6	10.50
ENet [12]	90.2	12.20	YOLOv7 [29]	88.4	5.20
Ours	95.0	14.63	Ours	88.1	14.63

IoU of 94.8%, outperforming BiSeNet [30] and ENet [12] by 2.7 and 4.6 percentage points, respectively. However, U-Net could not complete inference under the evaluated deployment configuration. Among the successfully deployed standalone models, ENet provides the highest throughput of 12.20 FPS, while BiSeNet achieves a higher IoU but operates at only 6.40 FPS.

For IoU computation, the lane outputs of our model are converted into binary lane regions at the ground-truth resolution. Ours achieves the highest overall IoU of 95.0%, exceeding U-Net by 0.2 percentage points. It also reaches 14.63 FPS, improving upon the fastest successfully deployed standalone model by 2.43 FPS, while jointly producing lane, object, and scene predictions. These results indicate that the shared multi-task representation preserves strong lane-perception accuracy while enabling efficient execution of the complete visual pipeline on the Jetson Nano.

3.4 Object-Detection Results

As shown in Tab. 2b, YOLOv7 [29] achieves the highest standalone mAP@0.5 of 88.4%, exceeding YOLOv5m [7] and YOLOv6n [8] by 2.3 and 2.8 percentage points, respectively. YOLOv6n provides the highest standalone throughput of 10.50 FPS but the lowest mAP, confirming the trade-off between speed and

localization accuracy. Ours achieves 88.1% mAP, only 0.3 percentage points below YOLOv7, while reaching the highest overall throughput of 14.63 FPS on the Jetson Nano. This is approximately 39.3% faster than YOLOv6n, despite concurrently producing object, lane, and scene predictions, demonstrating the computational benefit of shared feature extraction.

Onboard observations further show the complementary roles of the visual and LiDAR pipelines. The visual detector can recognize *small objects* when sufficient appearance cues are visible, whereas *closer objects* may occupy a *large portion of the image*, become truncated, and consequently be missed or misclassified. In such cases, LiDAR may still provide range returns from nearby surfaces intersecting its scanning plane, indicating occupied space and metric distance. However, these returns are not semantically classified or associated with visual bounding boxes; therefore, the reported mAP remains a camera-only metric and does not include any numerical improvement from LiDAR.

3.5 System-Level Task-Configuration Ablation

We compare the single-task baselines, their separate-network combination, and our shared multi-task model in Tab. 3. Because these configurations use different architectures, this experiment represents a system-level task-configuration ablation rather than a component-wise architectural ablation. The accuracy values for YOLOv7 + U-Net are inherited from the independently evaluated networks and do not represent joint training. YOLOv7 runs at 5.20 FPS, whereas the U-Net inference process crashed on the Jetson Nano under the evaluated configuration. Consequently, their separate-network combination could not execute onboard. In contrast, our shared model performs object detection, lane detection, and driving-scene classification within a single forward pass, achieving 88.1% mAP@0.5, 95.0% IoU, and 14.63 FPS on the Jetson Nano.

3.6 Tri-Modal System Evaluation

The final experiment examines the concurrent operation of the visual, LiDAR, and CAN pipelines on the Jetson Nano during the Renault Twizy deployment.

Table 3: **System-level task-configuration ablation.** Accuracy and onboard throughput of single-task baselines, their separate-network combination, and our shared multi-task model. Reported FPS values were measured on the NVIDIA Jetson Nano. For YOLOv7 + U-Net, the accuracy values are inherited from the independently evaluated single-task networks. *Failed* indicates an inference runtime failure on the Jetson Nano.

Configuration	mAP ₅₀ (%) ↑	IoU (%) ↑	FPS ↑
YOLOv7 (detection only)	88.4	–	<u>5.20</u>
U-Net (lane only)	–	<u>94.8</u>	<i>Failed</i>
YOLOv7 + U-Net	88.4	<u>94.8</u>	<i>Failed</i>
Ours (all tasks)	<u>88.1</u>	95.0	14.63

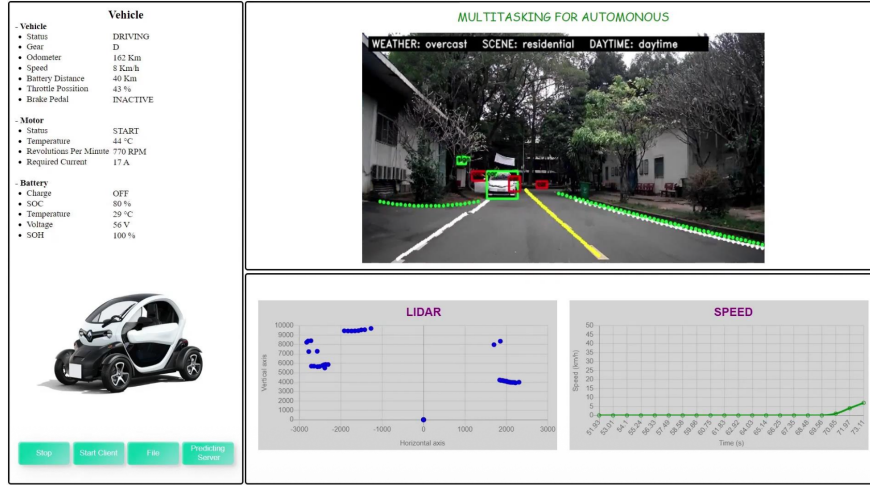


Fig. 6: **Tri-modal monitoring during the Renault Twizy experiment.** The interface jointly presents visual predictions, Cartesian LiDAR measurements, and CAN-derived vehicle states processed on the Jetson Nano.

As shown in Fig. 6, the monitoring interface presents object, lane, and scene predictions alongside metric LiDAR measurements and CAN-derived states, including vehicle speed, motor RPM, battery state, throttle position, and brake status. The experiment confirms that complementary visual, spatial, and vehicle-state information can be processed concurrently onboard.

The LiDAR and CAN pipelines complete each acquisition cycle in approximately 55 ms (18 updates/s) and 2.3 ms, respectively. These values characterize the individual sensing paths rather than the end-to-end system latency. Because the streams operate independently without sample-level synchronization, camera-LiDAR calibration, point-to-box association, or feature-level fusion, this evaluation demonstrates concurrent tri-modal integration and monitoring rather than sensor-level fusion.

4 Conclusion

We propose an **edge-based multi-task system with tri-modal perception** for autonomous driving. Unlike conventional pipelines executing separate networks, our approach employs a **shared visual representation** for lane perception, object detection, and scene classification, alongside independent LiDAR and CAN pipelines for environmental sensing and vehicle-state acquisition. This design reduces redundant computation and provides complementary semantic, spatial, and internal-state information on an NVIDIA Jetson Nano. Experiments in Unity, on a scale-model, and the Renault Twizy demonstrate the system preserves competitive performance while supporting concurrent processing of all modalities, establishing its practical feasibility on resource-constrained platforms.

References

1. Bui, K.P., Le, H.L.N., Le, Q.T., Huynh, D.H., Tran, V.H.: The design of a multi-task learning system for autonomous vehicles. In: 2022 6th International Conference on Green Technology and Sustainable Development (GTSD). pp. 201–207. IEEE (2022)
2. Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., Zagoruyko, S.: End-to-end object detection with transformers. In: Proceedings of the European Conference on Computer Vision (ECCV). pp. 213–229 (2020)
3. Cook, J.S.F.J.A.: Controller Area Network (CAN). University of Michigan (2008)
4. Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., Koltun, V.: CARLA: An open urban driving simulator. In: Proceedings of the 1st Annual Conference on Robot Learning. Proceedings of Machine Learning Research, vol. 78, pp. 1–16. PMLR (2017), <https://proceedings.mlr.press/v78/dosovitskiy17a.html>
5. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 770–778 (2016)
6. Howard, A., Sandler, M., Chu, G., Chen, L.C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q.V., Adam, H.: Searching for mobilenetv3. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). pp. 1314–1324 (2019)
7. Jocher, G.: YOLOv5 by ultralytics. Zenodo (2022). <https://doi.org/10.5281/zenodo.3908559>, <https://doi.org/10.5281/zenodo.3908559>, version 7.0
8. Li, C., Li, L., Jiang, H., Weng, K., Geng, Y., Li, L., Ke, Z., Li, Q., Cheng, M., Nie, W., Li, Y., Zhang, B., Liang, Y., Zhou, L., Xu, X., Chu, X., Wei, X., Wei, X.: YOLOv6: A single-stage object detection framework for industrial applications. arXiv preprint arXiv:2209.02976 (2022). <https://doi.org/10.48550/arXiv.2209.02976>
9. Liu, R.T., Yuan, Z., Liu, T., Lv, Z., Wang, X., Bai, X.: Conclanenet: A top-to-down lane detection framework based on conditional convolution. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 3773–3782 (2021)
10. Ma, N., Zhang, X., Zheng, H.T., Sun, J.: Shufflenet v2: Practical guidelines for efficient cnn architecture design. In: Proceedings of the European Conference on Computer Vision (ECCV). pp. 116–131 (2018)
11. National Transportation Safety Board: Collision between vehicle controlled by developmental automated driving system and pedestrian, tempe, arizona, march 18, 2018. Tech. Rep. NTSB/HAR-19/03, National Transportation Safety Board, Washington, DC (2019), <https://www.nts.gov/investigations/AccidentReports/Reports/HAR1903.pdf>
12. Paszke, A., Chaurasia, A., Kim, S., Culurciello, E.: ENet: A deep neural network architecture for real-time semantic segmentation. arXiv preprint arXiv:1606.02147 (2016). <https://doi.org/10.48550/arXiv.1606.02147>
13. Prykhodchenko, R., Skruch, P.: Road scene classification based on street-level images and spatial data. Array **15**, 100195 (2022)
14. Qian, Y., Dolan, J.M., Yang, M.: Dlt-net: Joint detection of drivable areas, lane lines, and traffic objects. IEEE Transactions on Intelligent Transportation Systems **21**(11), 4670–4679 (2019)
15. Qin, Z., Wang, H., Li, X.: Ultra fast structure-aware deep lane detection with focus on global information. In: Proceedings of the European Conference on Computer Vision (ECCV). pp. 321–336 (2020)

16. Renault UK: Renault twizy press kit. <https://www.press.renault.co.uk/assets/documents/original/14010-RenaultTwizyPressKitFebruary2018.pdf> (Feb 2018), accessed: September 2, 2026
17. Ronneberger, O., Fischer, P., Brox, T.: U-net: Convolutional networks for biomedical image segmentation. In: Medical Image Computing and Computer-Assisted Intervention (MICCAI). pp. 234–241 (2015)
18. Sakaridis, C., Dai, D., Van Gool, L.: Semantic foggy scene understanding with synthetic data. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 165–174 (2018)
19. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.C.: Mobilenetv2: Inverted residuals and linear bottlenecks. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 4510–4520 (2018)
20. Scribano, C., Franchini, G., Olmedo, I.S., Bertogna, M.: Cerberus: Simple and effective all-in-one automotive perception model with multi task learning. arXiv preprint arXiv:2210.00756 (2022)
21. SICK AG: LMS5xx Laser Measurement Sensors: Operating Instructions. SICK AG, Waldkirch, Germany (2014), part No. 8013777
22. Standley, T., Zamir, A., Chen, D., Guibas, L., Malik, J., Savarese, S.: Which tasks should be learned together in multi-task learning? In: Proceedings of the 37th International Conference on Machine Learning. pp. 9120–9132 (2020)
23. Tabelini, L., Berriel, R., Paixao, T.M., Badue, C., De Souza, A.F., Oliveira-Santos, T.: Keep your eyes on the lane: Real-time attention-guided lane detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 294–302 (2021)
24. Tan, M., Le, Q.V.: Efficientnet: Rethinking model scaling for convolutional neural networks. In: Proceedings of the International Conference on Machine Learning (ICML). pp. 6105–6114 (2019)
25. Tan, M., Pang, R., Le, Q.V.: Efficientdet: Scalable and efficient object detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 10781–10790 (2020)
26. Tang, J., Liu, S., Duo, Q., Lv, X.: Road scene classification based on convolutional neural networks and fisher vector. *IEEE Access* **7**, 65147–65155 (2019)
27. Teichmann, M., Weber, M., Zoellner, M., Cipolla, R., Urtasun, R.: Multinet: Real-time joint semantic reasoning for autonomous driving. In: 2018 IEEE intelligent vehicles symposium (IV). pp. 1013–1020. IEEE (2018)
28. Unity Technologies: Unity real-time development platform. <https://unity.com/>, accessed: September 4, 2026
29. Wang, C.Y., Bochkovskiy, A., Liao, H.Y.M.: YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 7464–7475 (2023)
30. Yu, C., Wang, J., Peng, C., Gao, C., Yu, G., Sang, N.: BiSeNet: Bilateral segmentation network for real-time semantic segmentation. In: Proceedings of the European Conference on Computer Vision. pp. 325–341 (2018)
31. Yu, T., Kumar, S., Gupta, A., Levine, S., Hausman, K., Finn, C.: Gradient surgery for multi-task learning. In: Advances in Neural Information Processing Systems. vol. 33, pp. 5824–5836 (2020)
32. Zang, S., Ding, M., Smith, D., Tyler, P., Rakotoarivelo, T., Kaafar, M.A.: The impact of adverse weather conditions on autonomous vehicles: How rain, snow, fog, and hail affect the performance of a self-driving car. *IEEE Vehicular Technology Magazine* **14**(2), 103–111 (2019)

33. Zheng, T., Huang, Y., Liu, Y., Tang, W., Yang, Z., Cai, D., He, X.: Clnet: Cross layer refinement network for lane detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 898–907 (2022)
34. Zhu, X., Su, W., Lu, L., Li, B., Wang, X., Dai, J.: Deformable detr: Deformable transformers for end-to-end object detection. In: International Conference on Learning Representations (ICLR) (2021)